Andromeda Spec 04: Rename

Oct 10, 2026 · @Jerry

Part of the Andromeda spec set. Shared types and conventions are defined in file 00.

Modules: `rename_stage`, `rat`, `rrat`, `free_list`, `busy_table` · **Milestone:** Instruction dispatch, issue and register renaming

1. What this unit does

Rename replaces architectural register names (x0 to x31, f0 to f31) with physical registers (64 per file). Every instruction that writes a register gets a fresh physical register, so instructions that only share a register *name* no longer have to wait for each other. Only true data dependences remain. Rename handles 4 instructions per cycle.

2. How it works

2.1 The three structures

Structure	Holds	Updated when
RAT (speculative)	For each architectural register, the physical register with its newest value	Every rename
rrat (committed)	The same mapping, but only as of the last committed instruction	Every commit
Free list	Physical registers not in use	Allocate at rename, release at commit
Busy table	One bit per physical register: 1 until its value is written	Set at rename, cleared at writeback

At reset, both tables hold the identity mapping (x1 → p1, … x31 → p31), the free list holds p32 to p63, and nothing is busy.

2.2 Renaming one instruction

For `add x3, x1, x4`:

1. **Sources are translated.** Look up x1 and x4 in the RAT to get their current physical registers, say p40 and p4. Look up each in the busy table to get its ready bit.
2. **The destination gets a new register.** Take one from the free list, say p41. Read the RAT's current mapping for x3 (say p3) and save it as `old_prd`. Write x3 → p41 into the RAT and mark p41 busy.

Sources are only looked up; only the destination allocates. Instructions with no destination (stores, branches, writes to x0) allocate nothing.

2.3 Four instructions at once

The RAT is updated at the end of the cycle, so within a group, later instructions can't see earlier ones' new mappings in the table. Rename fixes this with comparisons inside the group:

- **Sources:** if an earlier instruction in the group writes the register a later one reads, the later one takes the earlier one's new `prd` (and marks the source not ready, since it obviously hasn't been produced). If several earlier ones write it, the closest earlier one wins.
- **old_prd:** the same rule. If an earlier instruction in the group writes the same destination, the later one's `old_prd` is that earlier instruction's `prd`, not the table value.
- **Table write:** if two instructions in the group write the same register, only the later one's mapping goes into the RAT.

Example: `add x3, x1, x4` then `sub x5, x3, x2` in the same group. The `sub` must read the `add`'s new register for x3, not what the RAT says.

2.4 Freeing registers

A physical register can only be freed once nothing can read it again. That moment is when the *next* writer of the same architectural register commits. At that point every older instruction has retired, so nobody still needs the old value. That's why rename saves `old_prd`, the ROB stores it, and commit returns it to the free list.

2.5 Recovery after a flush

The rrat always holds the mapping as of the last committed instruction. On a flush:

- the RAT is overwritten with the rrat,
- the free list rewinds to its committed position (every register allocated by a discarded instruction becomes free again),
- every busy bit is cleared, because every register still mapped holds a committed, written value.

No per-instruction undo is needed.

2.6 x0 and f0

x0 is never renamed. It always maps to integer physical register 0, which reads as zero, is never busy and is never handed out by the free list. f0 is an ordinary register and renames like any other.

3. Interface

3.1 rename_stage

Signal	Dir	Width	Description
`flush`	in	1	`be_flush`
`in_valid` / `in_uop`	in	4 / 4 × uop_t	From decode
`in_ready`	out	1	Group accepted (enough free registers in both files)
`out_valid` / `out_uop`	out	4 / 4 × uop_t	Renamed uops: prs1/2/3, prd, old_prd

Signal	Dir	Width	Description
			and ready bits filled
out_ready	in	1	Dispatch took the group
wakeup	in	4 × wakeup_t	Clears busy bits; also checked against sources renamed this cycle
cm_valid	in	4	Retiring uops this cycle
cm_rd_en / cm_rd_fp / cm_rd_arch / cm_prd / cm_old_prd	in	4 × (1 / 1 / 5 / 7 / 7)	Updates rrat; old_prd returns to the free list

3.2 rat and rrat (one each, covering both files)

Signal	Dir	Width	Description
rd_fp / rd_arch	in	16 × (1 / 5)	12 source lookups and 4 old-destination lookups per cycle
rd_ptag	out	16 × 7	Current mapping
we / wr_fp / wr_arch / wr_ptag	in	4 × (1 / 1 / 5 / 7)	New mappings; slot 3 has priority on conflicts
restore	in	1	rat only: copy all 64 entries from rrat (driven by flush)
committed_map	out (rrat) / in (rat)	64 × 7	Full-table copy path

3.3 free_list (one per register file)

Signal	Dir	Width	Description
alloc_req	in	4	Slots needing a register in this file
alloc_ptag	out	4 × 6	Allocated indices
alloc_ok	out	1	At least as many free registers as requested
free_valid / free_ptag	in	4 / 4 × 6	Old registers released at commit
commit_alloc_cnt	in	3	Allocations that committed this cycle; advances the committed head
flush	in	1	Rewind allocation head to committed head

3.4 busy_table (one per register file)

Signal	Dir	Width	Description
set_valid / set_idx	in	4 / 4 × 6	Mark newly allocated registers busy

Signal	Dir	Width	Description
`clr_valid` / `clr_idx`	in	4 / 4 × 6	From wakeup buses
`rd_idx`	in	12 × 6	Source lookups
`rd_busy`	out	12	1 = value not yet produced
`flush`	in	1	Clear all busy bits

4. Implementation notes

- **RAT as flip-flops.** 64 entries of 7 bits is small, but 16 read ports means 16 copies of a 64-to-1 mux. Splitting into an integer half and an FP half (32 entries each) halves each mux, since a lookup already knows which file it wants.
- **Dependency check logic.** For 4 slots and 3 sources each, the intra-group check is a small matrix of 5-bit comparators (each source against the destinations of all earlier slots), followed by a priority pick of the closest match. Write it as a loop over slots in `always_comb`; it's easy to read and synthesizes to the expected structure. This is one of the timing-critical paths in the core.
- **Free list as a circular FIFO.** Hold 32 entries of free register indices with three pointers: allocation head, committed head and tail. Allocation pops from the head, commit pushes freed registers at the tail, and committed allocations advance the committed head. Flush sets the head back to the committed head. The entries between them still contain exactly the registers the discarded instructions took, so nothing else is needed. The shared multi-port FIFO (file 00) covers most of this.
- **Committed head bookkeeping.** `commit_alloc_cnt` counts retiring instructions that allocated in this file, which the ROB derives from `cm_rd_en` and `cm_rd_fp`. Keep the count and the freed registers consistent; a register leak here shows up much later as the core slowly running out of registers.
- **Same-cycle wakeup.** A source renamed in the same cycle its producer writes back must see the wakeup, or it records "busy" and waits forever. Compare source tags against the 4 wakeup buses in parallel with the busy table read. The same applies while a renamed group sits in the rename output register waiting for dispatch: keep updating its ready bits from the wakeup buses.
- **Stall granularity.** v1 stalls the whole group if either file lacks enough free registers. Simple, and rarely costly with 32 spare registers per file.

5. Verification checklist

- ☐ Unit test against a software model of the RAT, free list and busy table with random groups, commits and flushes
- ☐ Every intra-group pattern: RAW, WAW and both within one group, in every slot pair
- ☐ Writes to x0 never allocate; reads of x0 are always ready
- ☐ Register conservation: after any random sequence, every physical register is exactly one of free, committed-mapped or held by an in-flight instruction: 64 in the FP file, 63 in the integer file excluding p0 (an assertion is ideal)
- ☐ Flush restores RAT to rrat and rewinds the free list

- ☐ Wakeup in the same cycle as rename marks the source ready
- ☐ Stall when free registers run low; resumes after commits release registers